Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.01 Информатика и вычислительная техника

ОТЧЕТ

Лабораторная работа № 13

По дисциплине «Теория алгоритмов и структуры данных»

Вариант № 25

Выполнил: студент группы ИВТ-25-2б
Соловьев Тимофей Николаевич

Принял: Доц. Полякова О. А.

Пермь, 2026

1. Постановка задачи

```
Постановка задачи
Задача 1.
1. Создать последовательный контейнер. 2. Заполнить его элементами пользовательского типа (тип указан в
варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Заменить элементы в соответствии с заданием (использовать алгоритмы
replace_if(), replace_copy(), replace_copy_if(), fill()).
4. Удалить элементы в соответствии с заданием (использовать алгоритмы
remove(),remove_if(), remove_copy_if(),remove_copy())
5. Отсортировать контейнер по убыванию и по возрастанию ключевого поля
(использовать алгоритм sort()).
6. Найти в контейнере заданный элемент (использовать алгоритмы find(),
find_if(), count(), count_if()).
7. Выполнить задание варианта для полученного контейнера (использовать алгоритм
for_each()) .
8. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки
STL.
Задача 2.
1. Создать адаптер контейнера.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Заменить элементы в соответствии с заданием (использовать алгоритмы
replace_if(), replace_copy(), replace_copy_if(), fill()).
4. Удалить элементы в соответствии с заданием (использовать алгоритмы
remove(),remove_if(), remove_copy_if(),remove_copy())
5. Отсортировать контейнер по убыванию и по возрастанию ключевого поля
(использовать алгоритм sort()).
6. Найти в контейнере элемент с заданным ключевым полем (использовать
алгоритмы find(), find_if(), count(), count_if()).
7. Выполнить задание варианта для полученного контейнера (использовать алгоритм
for_each()) .
8. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки
STL.
Задача 3
1. Создать ассоциативный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Заменить элементы в соответствии с заданием (использовать алгоритмы
replace_if(), replace_copy(), replace_copy_if(), fill()).
4. Удалить элементы в соответствии с заданием (использовать алгоритмы
remove(),remove_if(), remove_copy_if(),remove_copy())
5. Отсортировать контейнер по убыванию и по возрастанию ключевого поля
(использовать алгоритм sort()).
6. Найти в контейнере элемент с заданным ключевым полем (использовать
алгоритмы find(), find_if(), count(), count_if()).
7. Выполнить задание варианта для полученного контейнера (использовать алгоритм
for_each()) .
8. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки
STL. адача 1
1. Контейнер - список
2. Тип элементов Pair (см. лабораторную работу No3).
Задача 2
Адаптер контейнера – очередь с приоритетами.
Задача 3
Ассоциативный контейнер - словарь . Задание 3 Найти среднее
арифметическое и
добавить его в конец
контейнера Задание 4 Найти элементы
ключами из заданного
диапазона и удалить их
из контейнера Задание 5 К каждому элементу добавить
сумму минимального и
максимального элементов
контейнера.
```

2. Анализ

```
Программа состоит из трех задач, демонстрирующих работу с различными контейнерами и адаптерами
стандартной библиотеки C++ с использованием алгоритмов STL.

В первой задаче используется последовательный контейнер list для хранения объектов пользовательского типа
Pair. Структура Pair содержит два целых поля first и second. Оператор меньше перегружен для сравнения
```

только по полю first, что определяет порядок сортировки. Оператор равенства позволяет сравнивать пары поэлементно. Оператор вывода выводит пару в круглых скобках через запятую.

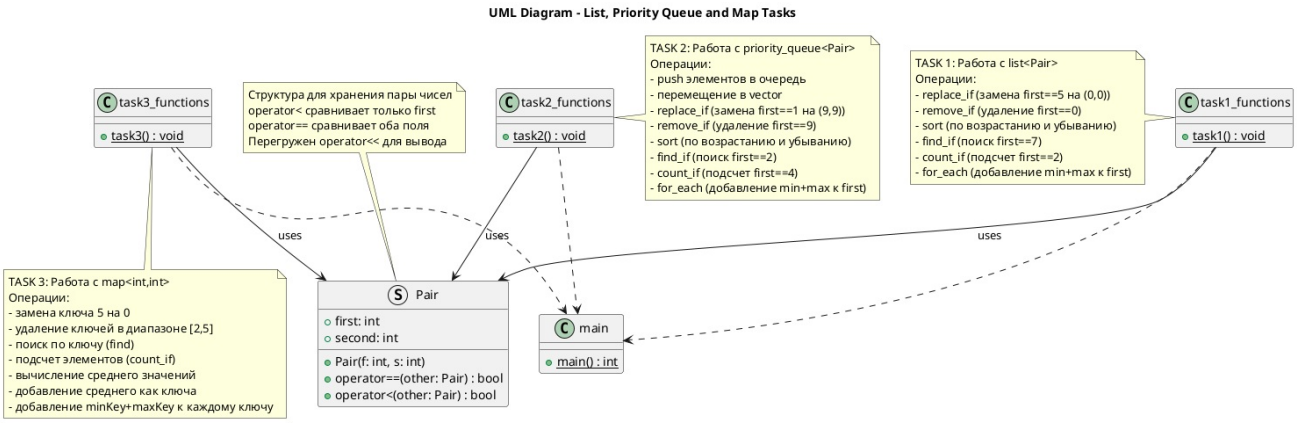
Исходный список содержит семь пар. Алгоритм replace_if заменяет все элементы, у которых поле first равно пяти, на пару с нулевыми значениями. Затем метод remove_if удаляет все элементы с нулевым полем first. Для сортировки используется метод sort списка с пользовательскими компараторами сначала по возрастанию, затем по убыванию поля first. Алгоритм find_if ищет первый элемент с полем first равным семи. Алгоритм count_if подсчитывает количество элементов с полем first равным двум. Для выполнения варианта задания используются алгоритмы minmax_element для нахождения минимального и максимального элементов по полю first и for_each для прибавления суммы минимального и максимального значений к полю first каждого элемента.

Во второй задаче используется адаптер контейнера priority_queue, настроенный как max-heap на основе поля first структуры Pair. В очередь помещаются пять пар. Для выполнения операций замены и удаления элементы извлекаются из очереди в вектор. Алгоритм replace_if заменяет все элементы с полем first равным единице на пару со значениями девять и девять. Алгоритм remove_if удаляет элементы с полем first равным девяти, после чего erase окончательно удаляет их из вектора. Сортировка выполняется алгоритмом sort с компараторами для возрастания и убывания по полю first. Алгоритм find_if ищет элемент с полем first равным двум. Алгоритм count_if подсчитывает элементы с полем first равным четырем. Для варианта задания находятся минимальный и максимальный элементы по полю first с помощью minmax_element, затем for_each прибавляет сумму этих значений к полю first каждого элемента.

В третьей задаче используется ассоциативный контейнер map, хранящий пары ключ-значение, где ключ и значение являются целыми числами. Исходный словарь содержит пять записей. Замена элемента выполняется созданием временного словаря, в который копируются все записи, кроме записи с ключом пять, вместо нее добавляется запись с ключом ноль. Удаление элементов в диапазоне ключей от двух до пяти включительно выполняется прямым обходом итератора с вызовом erase. Сортировка в map выполняется автоматически по возрастанию ключей, поэтому специальной сортировки не требуется. Поиск элемента по ключу выполняется методом find. Алгоритм count_if подсчитывает количество записей с положительным ключом. Для варианта задания вычисляется среднее арифметическое всех значений с помощью accumulate, затем в словарь добавляется новый элемент с ключом, равным целой части среднего, и значением ноль. Затем находятся минимальный и максимальный ключи через begin и rbegin, вычисляется их сумма, и создается новый словарь, в котором каждый ключ увеличен на эту сумму при сохранении соответствующих значений.

Основная функция последовательно вызывает все три задачи, каждая из которых выводит промежуточные и итоговые результаты своих операций. Программа демонстрирует применение стандартных алгоритмов STL для различных типов контейнеров.

3. UML-диаграмма



4. Исходный код

Файл реализации (.cpp)

```
#include
#include
#include
#include
#include
#include
#include

struct Pair {
    int first;
    int second;

    Pair(int f = 0, int s = 0) : first(f), second(s) {}

    bool operator==(const Pair& other) const {
```

```
        return first == other.first && second == other.second;
    }

    bool operator<(const Pair& other) const {
        return first < other.first;
    }
};

std::ostream& operator<<(std::ostream& os, const Pair& p) {
    os << "(" << p.first << "," << p.second << ")";
    return os;
}

void task1() {
    std::cout << "\n=== TASK 1: LIST ===\n";
    std::list lst = {{5,1}, {2,9}, {5,3}, {7,4}, {2,5}, {5,6}, {8,7}};
    std::cout << "Initial: "; for(auto& p : lst) std::cout << p << " "; std::cout << "\n";

    std::replace_if(lst.begin(), lst.end(), [](const Pair& p){ return p.first == 5; }, Pair(0,0));
    std::cout << "After replace_if (first==5 -> (0,0)): "; for(auto& p : lst) std::cout << p << " ";
std::cout << "\n";

    lst.remove_if([](const Pair& p){ return p.first == 0; });
    std::cout << "After remove_if (first==0): "; for(auto& p : lst) std::cout << p << " "; std::cout <<
"\n";

    lst.sort([](const Pair& a, const Pair& b){ return a.first < b.first; });
    std::cout << "Sorted ascending by first: "; for(auto& p : lst) std::cout << p << " "; std::cout <<
"\n";

    lst.sort([](const Pair& a, const Pair& b){ return a.first > b.first; });
    std::cout << "Sorted descending by first: "; for(auto& p : lst) std::cout << p << " "; std::cout <<
"\n";

    auto it = std::find_if(lst.begin(), lst.end(), [](const Pair& p){ return p.first == 7; });
    if(it != lst.end()) std::cout << "Found element with first==7: " << *it << "\n";

    int cnt = std::count_if(lst.begin(), lst.end(), [](const Pair& p){ return p.first == 2; });
    std::cout << "Count of elements with first==2: " << cnt << "\n";

    if(!lst.empty()) {
        auto [minIt, maxIt] = std::minmax_element(lst.begin(), lst.end(),
            [](const Pair& a, const Pair& b){ return a.first < b.first; });
        int sumMinMax = minIt->first + maxIt->first;
        std::for_each(lst.begin(), lst.end(), [sumMinMax](Pair& p){ p.first += sumMinMax; });
        std::cout << "After adding (min+max=" << sumMinMax << ") to each first: ";
        for(auto& p : lst) std::cout << p << " "; std::cout << "\n";
    }
}

void task2() {
    std::cout << "\n=== TASK 2: PRIORITY QUEUE ===\n";
    std::priority_queue pq;
    std::vector data = {{3,1}, {1,2}, {4,3}, {1,4}, {2,5}};

    for(auto& p : data) pq.push(p);
    std::cout << "Priority queue (max-heap by first): ";
    auto temp = pq;
    while(!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
    std::cout << "\n";

    std::vector vec;
    while(!pq.empty()) { vec.push_back(pq.top()); pq.pop(); }

    std::replace_if(vec.begin(), vec.end(), [](const Pair& p){ return p.first == 1; }, Pair(9,9));
    std::cout << "After replace_if (first==1 -> (9,9)): "; for(auto& p : vec) std::cout << p << " ";
std::cout << "\n";

    auto newEnd = std::remove_if(vec.begin(), vec.end(), [](const Pair& p){ return p.first == 9; });
    vec.erase(newEnd, vec.end());
    std::cout << "After remove_if (first==9): "; for(auto& p : vec) std::cout << p << " "; std::cout <<
"\n";

    std::sort(vec.begin(), vec.end(), [](const Pair& a, const Pair& b){ return a.first < b.first; });
    std::cout << "Sorted ascending: "; for(auto& p : vec) std::cout << p << " "; std::cout << "\n";

    std::sort(vec.begin(), vec.end(), [](const Pair& a, const Pair& b){ return a.first > b.first; });
    std::cout << "Sorted descending: "; for(auto& p : vec) std::cout << p << " "; std::cout << "\n";
```

```
    auto it = std::find_if(vec.begin(), vec.end(), [](const Pair& p){ return p.first == 2; });
    if(it != vec.end()) std::cout << "Found element with first==2: " << *it << "\n";

    int cnt = std::count_if(vec.begin(), vec.end(), [](const Pair& p){ return p.first == 4; });
    std::cout << "Count of elements with first==4: " << cnt << "\n";

    if(!vec.empty()) {
        auto [minIt, maxIt] = std::minmax_element(vec.begin(), vec.end(),
            [](const Pair& a, const Pair& b){ return a.first < b.first; });
        int sumMinMax = minIt->first + maxIt->first;
        std::for_each(vec.begin(), vec.end(), [sumMinMax](Pair& p){ p.first += sumMinMax; });
        std::cout << "After adding (min+max=" << sumMinMax << ") to each first: ";
        for(auto& p : vec) std::cout << p << " "; std::cout << "\n";
    }
}

void task3() {
    std::cout << "\n=== TASK 3: MAP (DICTIONARY) ===\n";
    std::map m = {{5,1}, {2,9}, {7,4}, {8,7}, {3,6}};
    std::cout << "Initial map: ";
    for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
    std::cout << "\n";

    std::map tempMap;
    for(auto& [k,v] : m) {
        if(k == 5) tempMap[0] = 0;
        else tempMap[k] = v;
    }
    m = tempMap;
    std::cout << "After replacing key 5 with (0,0): ";
    for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
    std::cout << "\n";

    auto it = m.begin();
    while(it != m.end()) {
        if(it->first >= 2 && it->first <= 5) it = m.erase(it);
        else ++it;
    }
    std::cout << "After removing keys in [2,5]: ";
    for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
    std::cout << "\n";

    std::cout << "Map is always sorted ascending by key: ";
    for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
    std::cout << "\n";

    auto found = m.find(8);
    if(found != m.end()) std::cout << "Found element with key 8: (" << found->first << "," << found-
>second << ")\n";

    int cnt = std::count_if(m.begin(), m.end(), [](const auto& p){ return p.first > 0; });
    std::cout << "Count of elements with key>0: " << cnt << "\n";

    if(!m.empty()) {
        int sum = std::accumulate(m.begin(), m.end(), 0, [](int s, const auto& p){ return s + p.second;
});
        double mean = static_cast(sum) / m.size();
        std::cout << "Arithmetic mean of values: " << mean << "\n";

        m[static_cast(mean)] = 0;
        std::cout << "After adding mean as new key: ";
        for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
        std::cout << "\n";
    }

    if(!m.empty()) {
        int minKey = m.begin()->first;
        int maxKey = m.rbegin()->first;
        int sumMinMaxKey = minKey + maxKey;

        std::map modified;
        for(auto& [k,v] : m) {
            modified[k + sumMinMaxKey] = v;
        }
        m = modified;
        std::cout << "After adding (minKey+maxKey=" << sumMinMaxKey << ") to each key: ";
        for(auto& [k,v] : m) std::cout << "(" << k << "," << v << ") ";
        std::cout << "\n";
    }
```

```
}

int main() {
    task1();
    task2();
    task3();
    return 0;
}
```

5. Результаты выполнения

```
=== TASK 1: LIST ===
Initial: (5,1) (2,9) (5,3) (7,4) (2,5) (5,6) (8,7)
After replace_if (first==5 -> (0,0)): (0,0) (2,9) (0,0) (7,4) (2,5) (0,0) (8,7)
After remove_if (first==0): (2,9) (7,4) (2,5) (8,7)
Sorted ascending by first: (2,9) (2,5) (7,4) (8,7)
Sorted descending by first: (8,7) (7,4) (2,9) (2,5)
Found element with first==7: (7,4)
Count of elements with first==2: 2
After adding (min+max=10) to each first: (18,7) (17,4) (12,9) (12,5)

=== TASK 2: PRIORITY QUEUE ===
Priority queue (max-heap by first): (4,3) (3,1) (2,5) (1,4) (1,2)
After replace_if (first==1 -> (9,9)): (4,3) (3,1) (2,5) (9,9) (9,9)
After remove_if (first==9): (4,3) (3,1) (2,5)
Sorted ascending: (2,5) (3,1) (4,3)
Sorted descending: (4,3) (3,1) (2,5)
Found element with first==2: (2,5)
Count of elements with first==4: 1
After adding (min+max=6) to each first: (10,3) (9,1) (8,5)

=== TASK 3: MAP (DICTIONARY) ===
Initial map: (2,9) (3,6) (5,1) (7,4) (8,7)
After replacing key 5 with (0,0): (0,0) (2,9) (3,6) (7,4) (8,7)
After removing keys in [2,5]: (0,0) (7,4) (8,7)
Map is always sorted ascending by key: (0,0) (7,4) (8,7)
Found element with key 8: (8,7)
Count of elements with key>0: 2
Arithmetic mean of values: 3.66667
After adding mean as new key: (0,0) (3,0) (7,4) (8,7)
After adding (minKey+maxKey=8) to each key: (8,0) (11,0) (15,4) (16,7)
```

6. Ответы на вопросы

```
нет вопросов
```